

Rapport TP - Plateforme d'évaluation loi de Fitts et analyse GOMS/Keystroke

Table des matières

1. Introduction
2. Environnement de développement
 1. Installation de Python et PyCharm
 2. Configuration de l'environnement virtuel
 3. Installation de PySide6
 4. Installation des dépendances et démarrage
3. Architecture générale de l'application
 1. Structure du projet
 2. Navigation entre les écrans
4. Implémentation de la plateforme loi de Fitts
 1. Description de l'expérience
 2. Interface utilisateur
 3. Analyse statistique
 4. Réglage dynamique des paramètres
5. Implémentation de la plateforme Keystroke
 1. Expérience de charge cognitive
 2. Expérience de navigation texte
6. Utilisation de PySide6 vs Qt en C++
 1. Principales différences syntaxiques
 2. Avantages et inconvénients
7. Style et ergonomie
 1. Utilisation des feuilles de style QSS
 2. Responsive design
8. Fonctionnalités manquantes et améliorations possibles
9. Conclusion

Introduction

Dans le cadre du module HM40 de l'UTBM, il nous a été demandé de développer une plateforme d'évaluation permettant d'expérimenter la loi de Fitts et l'analyse GOMS/Keystroke. Contrairement aux exigences initiales qui stipulaient l'utilisation de C++ avec la bibliothèque Qt, j'ai obtenu l'autorisation d'utiliser Python avec PySide6, un port de Qt pour Python.

Ce rapport détaille l'implémentation réalisée, les choix techniques, les difficultés rencontrées, ainsi que les différences entre l'utilisation de Qt en C++ et PySide6 en Python. N'ayant pas de base de code préexistante, j'ai dû développer l'ensemble de l'application depuis le début, ce qui explique que certaines fonctionnalités n'ont pas pu être implémentées dans le temps imparti.

Environnement de développement

Installation de Python et PyCharm

Pour ce projet, j'ai utilisé Python 3.9 et l'IDE PyCharm de JetBrains. Voici les étapes d'installation :

Installation de Python :

1. Téléchargement de Python 3.9 depuis le site officiel (<https://www.python.org/downloads/>)
2. Installation avec l'option "Add Python to PATH" activée
3. Vérification de l'installation via la commande `python --version` dans un terminal

Installation de PyCharm :

1. Téléchargement de PyCharm Community Edition depuis le site de JetBrains (<https://www.jetbrains.com/pycharm/download/>)
2. Installation avec les options par défaut
3. Lancement de PyCharm et création d'un nouveau projet

Configuration de l'environnement virtuel

L'utilisation d'un environnement virtuel permet d'isoler les dépendances du projet et d'éviter les conflits avec d'autres projets. PyCharm facilite grandement cette gestion :

1. Lors de la création du projet, j'ai sélectionné "New environment using Virtualenv"
2. PyCharm a automatiquement créé un dossier `venv` dans le répertoire du projet
3. L'activation de l'environnement virtuel se fait automatiquement dans le terminal intégré de PyCharm

Pour ceux qui souhaiteraient reproduire ce projet sans PyCharm, voici les commandes équivalentes :

```
# Création de l'environnement virtuel
```

```
python -m venv venv
```

```
# Activation sur Windows
```

```
venv\Scripts\activate
```

```
# Activation sur macOS/Linux
```

```
source venv/bin/activate
```

Installation de PySide6

PySide6 est le binding officiel de Qt pour Python maintenu par The Qt Company. Il s'agit d'une alternative à PyQt, avec une licence plus permissive (LGPL).

Installation via PyCharm :

1. Aller dans File > Settings > Project > Python Interpreter
2. Cliquer sur "+" pour ajouter un package
3. Rechercher "PySide6" et l'installer

Ou via le terminal avec pip :

```
pip install PySide6
```

J'ai également installé les dépendances suivantes pour les graphiques statistiques :

```
pip install matplotlib numpy
```

Installation des dépendances et démarrage

Pour faciliter l'installation des dépendances et le démarrage de l'application, suivez ces étapes après avoir configuré l'environnement virtuel :

Installation des dépendances

```
# Activation de l'environnement virtuel (si ce n'est pas déjà fait)
```

```
# Sur Windows
```

```
venv\Scripts\activate
```

```
# Sur macOS/Linux
```

```
source venv/bin/activate
```

```
# Installation des dépendances
```

```
pip install -r requirements.txt
```

Alternativement, vous pouvez installer directement les packages principaux :

```
# Installation des packages essentiels
```

```
pip install PySide6 matplotlib numpy
```

Démarrage de l'application Une fois les dépendances installées, vous pouvez lancer l'application :

```
# Lancement de l'application (le point d'entrée est main.py)
```

```
python main.py
```

```
# Alternativement (si vous avez plusieurs versions de Python installées)
```

```
python3 main.py
```

Remarques importantes :

- Assurez-vous d'avoir Python 3.9 ou supérieur installé sur votre système
- L'environnement virtuel doit être activé à chaque fois que vous souhaitez travailler sur le projet (Sur Pycharm, il suffit d'ouvrir le terminal intégré)

- Si vous rencontrez des problèmes d’affichage avec l’interface graphique, vérifiez que PySide6 est correctement installé

Architecture générale de l’application

Structure du projet

L’application est structurée autour d’une fenêtre principale (**MainWindow**) qui gère plusieurs écrans. Cette approche modulaire facilite la navigation et la maintenance du code. Voici les principaux fichiers et leur rôle :

- **main.py** : Point d’entrée de l’application, initialise la fenêtre principale et configure l’interface utilisateur
- **constants.py** : Définit les constantes utilisées dans l’application, notamment les paramètres par défaut pour la loi de Fitts
- **menu_screen.py** : Écran de menu principal permettant de choisir entre les expériences
- **fitts_experiment_screen.py** : Écran pour l’expérience de Fitts
- **experiment_area.py** : Widget pour l’affichage et la manipulation des cibles dans l’expérience de Fitts
- **stats_screen.py** : Écran d’affichage des statistiques après une expérience
- **settings_screen.py** : Écran de configuration des paramètres de l’application
- **keystroke_menu_screen.py** : Menu pour les expériences de type keystroke
- **cognitive_load_experiment.py** : Expérience sur l’impact de la charge cognitive sur la performance de frappe
- **navigation_experiment.py** : Expérience comparant l’efficacité de navigation entre clavier et souris
- **style.qss** : Feuille de style pour l’application (équivalent CSS pour Qt)

Navigation entre les écrans

La navigation entre les différents écrans est gérée par un **QStackedWidget** dans la classe **MainWindow**. Chaque écran est ajouté au widget empilé avec un index, et la méthode **switch_screen(index)** permet de passer d’un écran à l’autre.

```
def switch_screen(self, index: int) -> None:
    """
    Switch the visible screen.

    Args:
        index (int): The index of the screen in the QStackedWidget.
    """
    self.stack.setCurrentIndex(index)
```

Cette approche présente plusieurs avantages :

- Les écrans sont préchargés au démarrage de l'application, ce qui permet une transition rapide
- L'état de chaque écran est conservé lorsqu'on navigue entre eux
- La navigation peut être contrôlée de manière centralisée

Implémentation de la plateforme loi de Fitts

Description de l'expérience

La loi de Fitts, formulée par Paul Fitts en 1954, modélise le temps nécessaire pour atteindre une cible en fonction de la distance à parcourir et de la taille de la cible. La formule utilisée est :

$$T = a + b * \log_2(D/W + 1)$$

Où :

- T est le temps pour atteindre la cible
- D est la distance à la cible
- W est la largeur de la cible
- a et b sont des coefficients empiriques

L'expérience implémentée consiste à générer des cibles de taille aléatoire à différentes positions, et à mesurer le temps que met l'utilisateur pour les atteindre avec la souris. Ces mesures sont ensuite comparées aux prédictions théoriques de la loi de Fitts.

Interface utilisateur

L'interface de l'expérience de Fitts comprend :

- Un bouton "Start" pour démarrer l'expérience
- Une zone d'expérimentation où les cibles apparaissent
- Des instructions claires pour l'utilisateur

La classe `FittsExperimentScreen` gère l'interface utilisateur, tandis que la classe `ExperimentArea` s'occupe de l'affichage et de la gestion des cibles.

```
def _init_ui(self) -> None:
    layout = QVBoxLayout()
    instructions = QLabel("Click Start to begin the Fitts experiment")
    instructions.setAlignment(Qt.AlignCenter)
    layout.addWidget(instructions)

    button_layout = QHBoxLayout()
    self.start_button = QPushButton("Start")
    self.start_button.clicked.connect(self.start_experiment)
    button_layout.addWidget(self.start_button)

    layout.addLayout(button_layout)
```

```

from experiment_area import ExperimentArea
self.area = ExperimentArea(self)
layout.addWidget(self.area, stretch=1)

self.setLayout(layout)

```

Analyse statistique

Une fois l'expérience terminée, les résultats sont affichés dans l'écran StatsScreen. Cette classe utilise Matplotlib pour générer des graphiques permettant de visualiser :

- Les temps réels vs les temps prédits par la loi de Fitts
- L'évolution des temps en fonction de l'indice de difficulté ($\log_2(D/W + 1)$)

Les graphiques sont intégrés dans l'interface Qt grâce à la classe FigureCanvasQTAgg de Matplotlib.

```

# Classe StatsScreen
def update_plot(self, *_):
    """
    Update the plot based on the current x-axis selection.
    Only shows regression line for Index of Difficulty view.
    """
    self.ax.clear()
    if not self.data:
        return

    x_axis_choice = self.x_axis_selector.currentText()

    if x_axis_choice == "Target Number":
        x_data = [d['target_index'] for d in self.data]
        x_label = "Target Number"
        # Connected points for time series views
        self.ax.plot(x_data, [d['actual_time'] for d in self.data], 'o-', label="Actual Time")
        self.ax.plot(x_data, [d['expected_time'] for d in self.data], 's--', label="Expected Time")
        # ... (autres types de visualisations)

```

Réglage dynamique des paramètres

Une fonctionnalité clé de l'application est la possibilité d'ajuster dynamiquement les paramètres a et b de la loi de Fitts, et de voir immédiatement l'impact sur les graphiques. Cela permet à l'utilisateur de trouver les valeurs qui correspondent le mieux aux données expérimentales.

```

# Dans la classe SettingsScreen
def apply_coefficients(self) -> None:
    """
    Apply just the coefficient changes without saving other settings.
    This allows the coefficients to affect the graph in real-time.
    """
    Settings.a_coefficient = self.coef_a.value()
    Settings.b_coefficient = self.coef_b.value()

    # If stats screen has data, update it with the new coefficients
    if hasattr(self.main_window, 'stats_screen') and self.main_window.stats_screen.data:
        # Recalculate expected times with new coefficients
        for data_point in self.main_window.stats_screen.data:
            d = data_point['D']
            w = data_point['W']
            difficulty = data_point['difficulty']
            data_point['expected_time'] = Settings.a_coefficient + Settings.b_coefficient *

        # Update the plot
        self.main_window.stats_screen.set_data(self.main_window.stats_screen.data,
                                                self.main_window.stats_screen.is_final_view)

```

Implémentation de la plateforme Keystroke

Expérience de charge cognitive

Cette expérience, implémentée dans la classe `CognitiveLoadExperiment`, mesure l'impact d'une tâche secondaire (répondre à des bips sonores) sur la performance de frappe. L'utilisateur doit effectuer deux sessions de frappe :

1. Une session de référence où l'utilisateur tape un texte à son rythme normal
2. Une session avec double tâche où l'utilisateur doit taper le même texte tout en répondant à des bips aléatoires en appuyant sur la touche Entrée

Les métriques collectées incluent :

- La vitesse de frappe (mots par minute)
- Le taux d'erreur
- Les temps de réponse aux bips

Cette expérience permet d'estimer l'impact d'une charge cognitive supplémentaire sur la performance de frappe, ce qui est utile pour comprendre les coûts mentaux des tâches interrompues dans le modèle GOMS/Keystroke.

```

def complete_current_phase(self) -> None:
    """
    Complete the current phase and proceed to the next phase or end experiment.
    """
    end_time = time.time()

```

```

phase_duration = end_time - self.phase_start_time

# Stop timers
self.beep_timer.stop()

# Calculate results
results = self.calculate_results(phase_duration)

# Store results based on current phase
if self.current_phase == "baseline":
    self.baseline_results = results
    # Move to break between phases
    self.start_break()
elif self.current_phase == "dual_task":
    self.dual_task_results = results
    # ... (vérification et fin de l'expérience)

```

Expérience de navigation texte

L'expérience de navigation, implémentée dans la classe `NavigationExperiment`, compare l'efficacité du clavier et de la souris pour naviguer dans un texte. L'utilisateur effectue plusieurs tâches de navigation (par exemple, aller au début d'une ligne spécifique, sélectionner un mot particulier) en utilisant uniquement le clavier, puis uniquement la souris.

Cette expérience permet d'évaluer empiriquement les opérateurs H (Homing) et K (Keystroke) du modèle GOMS/Keystroke, en mesurant le temps nécessaire pour passer de la souris au clavier et vice versa, ainsi que le temps pour effectuer des frappes de touches.

Les résultats sont présentés sous forme de graphiques comparatifs, montrant les différences de performance entre le clavier et la souris pour chaque type de tâche.

```

def generate_performance_graphs(self, task_comparisons: list) -> None:
    """
    Generate performance comparison graphs between keyboard and mouse navigation.

    Args:
        task_comparisons: List of dictionaries containing comparison data for each task
    """
    if not task_comparisons:
        return

    # Clear existing plots
    for ax in self.axes:
        ax.clear()

```



```

# Extract data for plotting
task_names = [comp["task_name"] for comp in task_comparisons]
task_descriptions = [comp["task_description"] for comp in task_comparisons]
keyboard_times = [comp["keyboard_time"] for comp in task_comparisons]
mouse_times = [comp["mouse_time"] for comp in task_comparisons]

# ... (génération des graphiques)

```

Utilisation de PySide6 vs Qt en C++

Principales différences syntaxiques

L'utilisation de PySide6 en Python présente plusieurs différences notables par rapport à Qt en C++ :

1. **Déclaration des classes** : En Python, pas besoin de déclarer les signaux et slots séparément comme en C++.

En C++ avec Qt :

```

class MyWidget : public QWidget {
    Q_OBJECT
public:
    MyWidget(QWidget *parent = nullptr);
public slots:
    void handleButton();
signals:
    void valueChanged(int value);
};

```

En Python avec PySide6 :

```

class MyWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

    def handleButton(self):
        # Implémentation

```

2. **Connexion des signaux et slots** : Syntaxe plus simple et intuitive en Python.

En C++ avec Qt :

```

connect(button, SIGNAL(clicked()), this, SLOT(handleButton()));

```

En Python avec PySide6 :

```

button.clicked.connect(self.handleButton)

```

3. **Type hints** : Python permet d'utiliser des annotations de type pour améliorer la lisibilité du code.

```
def switch_screen(self, index: int) -> None:
    self.stack.setCurrentIndex(index)
```

4. **Gestion de la mémoire** : Python gère automatiquement la mémoire grâce au garbage collector, contrairement à C++ où il faut être plus vigilant.

Avantages et inconvénients

Avantages de PySide6 par rapport à Qt en C++ :

- Développement plus rapide grâce à la syntaxe concise de Python
- Pas besoin de compilation, ce qui accélère le cycle de développement
- Gestion automatique de la mémoire
- Intégration facile avec d'autres bibliothèques Python (comme Matplotlib pour les graphiques)
- Prototypage rapide d'interfaces

Inconvénients :

- Performances légèrement inférieures pour les applications complexes
- Documentation moins complète que celle de Qt en C++
- Dépendance à l'interpréteur Python
- Taille des exécutables plus importante lors de la création d'applications autonomes

Style et ergonomie

Utilisation des feuilles de style QSS

Pour améliorer l'esthétique de l'application, j'ai utilisé une feuille de style QSS (Qt Style Sheets), qui est l'équivalent du CSS pour les applications Qt. Le fichier `style.qss` définit l'apparence de tous les widgets de l'application.

```
/* Modern sleek style for the application */
QWidget {
    background: #1a1a1a;
    color: #f3f3f3;
    font-family: 'Arial', 'Helvetica Neue', 'Liberation Sans', sans-serif;
    font-size: 15px;
}

QLabel#TitleLabel {
    font-size: 26px;
    font-weight: bold;
    margin-bottom: 32px;
```

```

        color: #f3f3f3;
    }

QPushButton {
    background: #232a34;
    color: #f3f3f3;
    border: none;
    border-radius: 22px;
    padding: 32px 0;
    font-size: 22px;
    font-weight: 600;
    margin: 12px;
    qproperty-iconSize: 48px 48px;
}

```

L'application du style se fait dans le fichier `main.py` :

```

if __name__ == "__main__":
    app = QApplication(sys.argv)
    # Apply modern stylesheet
    with open("style.qss", "r") as f:
        app.setStyleSheet(f.read())
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```

La feuille de style QSS permet de créer une interface moderne et cohérente, avec un thème sombre qui réduit la fatigue visuelle.

Responsive design

Bien que Qt offre nativement une certaine adaptabilité, j'ai pris soin d'implémenter un design responsive pour que l'application s'adapte correctement à différentes tailles d'écran et facteurs d'échelle.

Cela a été réalisé en utilisant :

- Des layouts flexibles (QVBoxLayout, QHBoxLayout) qui s'adaptent automatiquement
- Des politiques de taille (setSizePolicy) pour contrôler comment les widgets se redimensionnent
- Des facteurs d'étirement (stretch) pour allouer l'espace disponible

```

# Exemple dans stats_screen.py
self.canvas.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
layout.addWidget(self.canvas, stretch=2)

```

Fonctionnalités manquantes et améliorations possibles

En raison des contraintes de temps liées à la réimplémentation complète de l'application, certaines fonctionnalités n'ont pas pu être implémentées :

1. **Mode clair/sombre** : Actuellement, seul le mode sombre est disponible. Une amélioration serait d'ajouter un bouton pour basculer entre les modes clair et sombre.
2. **Sauvegarde des résultats** : Les résultats des expériences ne sont pas sauvegardés entre les sessions. Une fonction d'export en CSV ou JSON serait utile.
3. **Évaluation complète des opérateurs GOMS** : L'évaluation des opérateurs M (Mental) et P (Pointing) n'est pas complètement implémentée.
4. **Aide contextuelle** : Des bulles d'aide et une documentation intégrée amélioreraient l'expérience utilisateur.
5. **Internationalisation** : Supporter plusieurs langues rendrait l'application plus accessible.

Améliorations possibles :

1. **Optimisation des performances** : Certaines opérations, notamment lors de l'affichage des graphiques, pourraient être optimisées.
2. **Interface utilisateur plus riche** : Ajouter des animations, des transitions et des retours visuels plus élaborés.
3. **Tests unitaires** : Implémenter une suite de tests complète pour garantir la stabilité de l'application.
4. **Création d'un installateur** : Packaging de l'application pour une distribution plus facile.

Conclusion

Ce projet m'a permis de mettre en pratique les concepts théoriques vus en cours sur l'interaction homme-machine, notamment la loi de Fitts et le modèle GOMS/Keystroke. Malgré les défis liés à la réimplémentation complète de l'application en Python avec PySide6, j'ai réussi à développer une plateforme fonctionnelle permettant de réaliser des expériences sur ces deux aspects.

L'utilisation de Python avec PySide6 s'est avérée être un choix judicieux pour ce type de projet, offrant un bon équilibre entre facilité de développement et fonctionnalités. Les principales différences avec Qt en C++ concernent la syntaxe, plus concise en Python, et la gestion automatique de la mémoire.

Les expériences implémentées permettent de valider empiriquement les modèles théoriques et d'explorer les facteurs qui influencent l'efficacité de l'interaction homme-machine. Les résultats visuels sous forme de graphiques facilitent

l'interprétation des données et la comparaison entre les performances réelles et théoriques.

Bien que toutes les fonctionnalités initialement prévues n'aient pas pu être implémentées, l'application constitue une base solide qui pourrait être étendue dans le futur pour inclure d'autres types d'expériences et d'analyses.